

PDF RAG Assistant

A Retrieval-Augmented Generation (RAG) application that allows users to chat with their own PDF documents using Large Language Models (LLMs).

Features

- Chat with any PDF document
- Persistent vector database using ChromaDB
- Semantic search with embeddings
- Fast document retrieval
- Gemini-powered question answering
- FastAPI backend
- Streamlit web interface
- Local vector storage to avoid reprocessing documents

Tech Stack

AI & RAG: LangChain, Google Gemini 2.5 Flash, Hugging Face Embeddings, ChromaDB

Backend: FastAPI, Uvicorn

Frontend: Streamlit

Document Processing: PyPDFLoader, RecursiveCharacterTextSplitter

How It Works

1. Load a PDF document.
2. Split the document into chunks.
3. Generate embeddings for each chunk.
4. Store embeddings in ChromaDB.
5. Retrieve relevant chunks based on user queries.
6. Send the retrieved context to Gemini.
7. Generate an answer grounded in the document.

Example Use Cases

- Research papers
- Books and eBooks
- University notes
- Technical documentation
- Company manuals
- Reports and contracts
- Personal knowledge bases

Learning Objectives

- Retrieval-Augmented Generation (RAG)
- Embeddings and semantic search
- Vector databases
- LangChain workflows
- API development
- Frontend integration
- LLM application development

Future Improvements

- Multi-document support
- Chat memory
- Source citations
- PDF upload from UI
- User authentication
- Docker deployment
- Cloud hosting
- Hybrid search
- Advanced RAG pipelines

Author

Hooman Eskandari — Built as a personal learning project to explore Retrieval-Augmented Generation (RAG), vector databases, semantic search, and modern AI application development.